Manual técnico

Proyecto de Análisis y Diseño de Algoritmos 2026-B, Grupo 3 (Ruta), primera entrega

Alejandro Duque Muñoz Carlos Felipe Lema Ríos
Universidad de Caldas

Versión `entrega1-1.1`, 6 de noviembre de 2026

1 Qué hace el programa

Dado un caso en el formato JSON de la profesora, el programa resuelve dos problemas y deja la salida, el tiempo y la traza de cada uno:

- **Parte 1, aceleración.** Un proceso tiene n etapas con duración t_i. Acelerar la etapa i cuesta c_i y reduce su duración en d_i. Con un presupuesto B, se escogen las etapas a acelerar para que la duración total sea mínima.
- **Parte 2, auditoría.** Cada etapa, si se audita, cubre un conjunto de categorías de riesgo. Se escoge el menor número de etapas que cubra todas las categorías.

La formalización, las pruebas de NP-dureza y el análisis de eficiencia están en el informe (`docs/Informe_Tecnico.pdf`). Este manual explica cómo está hecho el programa y cómo usarlo. Informe, código, manual y resultados corresponden todos a la versión `entrega1-1.1`.

2 Algoritmos

2.1 Parte 1 cuando el flujo es una cadena: programación dinámica

Idea. Si las etapas van en serie, la duración total es la suma de las duraciones, así que minimizar la duración es maximizar el ahorro $\sum d_i x_i$ sin pasar de B en costo. Eso es la mochila 0/1.

La tabla $DP[i][b]$ guarda el mayor ahorro usando solo las primeras i etapas con presupuesto b:

$$DP[i][b] = \max(DP[i-1][b],\ DP[i-1][b-c_i]+d_i) \text{ si } c_i \le b, \qquad DP[i][b] = DP[i-1][b] \text{ si } c_i > b.$$

Para recuperar las etapas se recorre la tabla desde (n, B) hacia atrás: la etapa i se acelera si hacerlo mejoró *estrictamente* el ahorro. Ante un empate no se acelera. El tiempo es $\Theta(n \cdot B)$ y el espacio $O(n \cdot B)$.

Ejemplo. $n = 4$, $B = 7$, $t = (10, 8, 12, 6)$, $c = (4, 3, 5, 2)$, $d = (5, 4, 7, 3)$. La tabla es:

$i \setminus b$	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1 ($c=4, d=5$)	0	0	0	0	5	5	5	5
2 ($c=3, d=4$)	0	0	0	4	5	5	5	9
3 ($c=5, d=7$)	0	0	0	4	5	7	7	9
4 ($c=2, d=3$)	0	0	3	4	5	7	8	10

El ahorro óptimo es $DP[4][7] = 10$ y las etapas aceleradas son E03, E04. La duración baja de 36 a 26.

2.2 Parte 1 cuando el flujo tiene rutas paralelas: ramificación y acotamiento

Idea. Con rutas paralelas la duración es la del camino más largo, y acelerar una etapa que no está en el camino crítico no ayuda. El programa recorre un árbol de decisiones (acelerar o no cada etapa candidata, primero las de mayor ahorro). En cada nodo calcula dos cotas inferiores de la duración final y poda el nodo si ninguna mejora la mejor solución conocida. Parte de una solución inicial voraz. La búsqueda es exacta, pero en el peor caso exponencial; hay un tope de nodos (`--limite-nodos`) y, si se alcanza, la salida lo avisa con `"optimo_probado": false`.

El programa elige solo el método: usa el DP si el flujo es una cadena y ramificación y acotamiento en cualquier otro caso. El orden de las etapas sale de las aristas de `flujo`, no del orden de la lista.

2.3 Parte 2: voraz de cobertura

Idea. Mientras falten categorías, se elige la etapa que cubre más categorías todavía no cubiertas. Si hay empate gana la que aparece primero en el archivo, así que el resultado siempre es el mismo para la misma entrada. Antes de empezar se comprueba que cada categoría la cubra alguna etapa; si no, la instancia se reporta como infactible y se nombran las categorías sin cubrir. El voraz no garantiza el óptimo, pero queda a un factor $H(d) \leq \ln r + 1$ de él. Cuesta $O(n \cdot r \cdot \min(n, r))$.

Ejemplo. Con $r = 6$ y los conjuntos $S_1 = \{1, 2, 3, 4\}$, $S_2 = \{1, 3, 5\}$, $S_3 = \{2, 4, 6\}$, el voraz elige S_1 (cubre 4), luego S_2 (cubre la 5) y luego S_3 (cubre la 6): 3 interacciones. El óptimo es $\{S_2, S_3\}$, con 2.

3 Estructura del código

```
ada-g3-entrega1/
  README.md               instrucciones rápidas
  requirements.txt        dependencias (solo para las pruebas)
  ejecutar_casos.py       programa principal: corre los casos y guarda resultados
  generar_casos.py        genera los casos propios (casos_propios/)
  src/ada/
    entrada.py            lectura y validación del JSON
    grafo.py              orden topológico, detección de cadena, camino más largo
    aceleracion.py        Parte 1: DP para cadenas y ramificación y acotamiento
    auditoria.py          Parte 2: factibilidad, voraz y óptimo exacto (verificación)
    verificadores.py      verificación independiente de las soluciones
    medicion.py           tiempo por caso (mediana de repeticiones)
    errores.py            errores con mensajes claros
```

```
tests/                  pruebas automáticas (pytest)
casos/                  casos de la profesora (no se editan)
casos_propios/          casos propios generados
resultados/             salida, tiempo y traza de los casos de la profesora
resultados_propios/     lo mismo para los casos propios
docs/                   informe, manual y generador de tablas
```

Cada algoritmo está en su propio módulo y no lee archivos ni imprime: recibe una instancia ya validada y devuelve diccionarios. Leer y escribir archivos es trabajo de `ejecutar_casos.py`. Los verificadores no usan el código de los algoritmos: releen el JSON original y recalculan todo.

4 Formatos

4.1 Entrada

Un archivo JSON con estas claves; lo que no aparece en la tabla se ignora.

Clave	Contenido
`parametros.presupuesto_aceleracion_B`	entero $B \geq 0$ (obligatorio)
`parametros.numero_etapas`, `numero_categorias_riesgo`	opcionales; si no coinciden con las listas se advierte
`flujo` (o `caminos`)	lista de aristas `{"origen", "destino"}` (o `desde`/`hasta`); debe ser acíclico
`etapas` (o `estaciones`)	lista de etapas, cada una con `id`, `duracion_normal` ≥ 0, `costo_aceleracion` ≥ 0, `reduccion_tiempo_si_acelera` con $0 \leq d_i \leq t_i$ y `riesgos_cubiertos_auditoria_completa` (lista de ids de categoría)
`etapas[].variaciones_auditoria`	opcional; se lee pero no se usa, porque se usa la auditoría completa
`categorias_riesgo`	lista de `{id, categoria, importancia}`; si falta, se usa la unión de las listas de las etapas

Los números deben ser enteros. Cualquier problema produce un mensaje que dice qué campo falla, por ejemplo: `etapa E07: reduccion_tiempo_si_acelera (30) no puede superar duracion_normal (24)` o `el flujo tiene un ciclo, así que no hay un orden de etapas`. Un caso con error no detiene a los demás: se guarda en `error.json` y la ejecución termina con código 1.

4.2 Salida de la Parte 1 (`parte1_salida.json`)

```
{
  "version": "entrega1-1.1",
  "caso": "entrada_G3_mochila_20_nodos",
  "metodo": "programacion_dinamica_cadena",
  "presupuesto_B": 72,
  "etapas_aceleradas": ["E02", "E03", "E04", "E06", "E08", "E12"],
  "costo_usado": 72,
  "duracion_sin_acelerar": 501,
  "duracion_final": 446,
```

```
  "ahorro": 55,
  "optimo_probado": true,
  "x": { "E01": 0, "E02": 1, "... una entrada por etapa ...": 0 }
}
```

`ahorro` es la reducción de la duración total (en una cadena es $\sum d_i x_i$) y `x` indica, por etapa, si se acelera.

4.3 Salida de la Parte 2 (`parte2_salida.json`)

```
{
  "version": "entrega1-1.1",
  "caso": "entrada_G3_mochila_20_nodos",
  "metodo": "voraz_mayor_ganancia",
  "n_candidatos": 20,
  "r_categorias": 12,
  "factible": true,
  "interacciones_elegidas": ["E01", "E16", "E06", "E03"],
  "tamano": 4,
  "d_max_categorias_por_candidato": 4,
  "cubre_todas_las_categorias": true
}
```

`interacciones_elegidas` está en el orden en que las eligió el voraz. Si la instancia es infactible, el archivo trae `"factible": false`, un `mensaje` y `categorias_sin_cubrir`.

4.4 Trazas

- `parte1_traza.json`, cadena: `decisiones_reconstruccion` (por etapa: presupuesto antes, $DP[i][b]$, decisión, costo, ahorro, presupuesto después), `dimension_tabla` y la tabla `tabla_dp` completa si tiene hasta 60 000 celdas (si es mayor, se omite y se avisa).
- `parte1_traza.json`, flujo general: nodos explorados, podas por cota, duración de la solución inicial y la lista de mejoras de la mejor solución.
- `parte2_traza.json`: una entrada por vuelta del voraz con la etapa elegida, su ganancia, las categorías nuevas, las cubiertas acumuladas, cuántas y cuáles faltan, las etapas que empataron con la elegida y perdieron por el desempate, y los cinco mejores candidatos de esa vuelta.
- `traza_legible.txt`: las dos trazas en texto, para leerlas sin abrir los JSON.

4.5 Tiempos y verificación

```
{
  "version": "entrega1-1.1",
  "caso": "entrada_G3_mochila_20_nodos",
  "parte1": {"repeticiones": 15, "mediana_ms": 0.0731, "minimo_ms": 0.0653, "maximo_ms": 0.1258},
  "parte2": {"repeticiones": 15, "mediana_ms": 0.0366, "minimo_ms": 0.031, "maximo_ms": 0.0466}
}
```

El tiempo es el del algoritmo solo (sin leer el archivo ni escribir resultados), con la mediana de varias repeticiones (7 por defecto). `verificacion.json` dice, para cada parte, si la solución respeta las restricciones, si las cifras coinciden con las recalculadas y, cuando el tamaño lo permite, si es óptima según una enumeración exhaustiva. `resumen.md`, `resumen.csv` y `resumen.json` resumen todos los casos.

5 Pasos de ejecución

Se necesita Python 3 (probado con la versión 3.13). El programa no usa librerías externas; `pytest` solo se necesita para correr las pruebas.

1. Abrir una terminal en la carpeta del repositorio.
2. (Opcional, solo para las pruebas) crear un entorno virtual e instalar las dependencias:

```
python -m venv .venv
source .venv/bin/activate          # en Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

3. Correr los casos de la profesora (están en `casos/`); los resultados quedan en `resultados/`:

```
python ejecutar_casos.py
```

4. Correr un solo caso, o los casos propios, o cambiar la carpeta de salida:

```
python ejecutar_casos.py casos/entrada_G3_mochila_20_nodos.json
python ejecutar_casos.py casos_propios --salida resultados_propios
python ejecutar_casos.py mis_casos/ --salida mis_resultados --repeticiones 3
```

5. Opciones útiles: `--limite-nodos N` (tope del solver general), `--sin-verificar` (omite los verificadores).
6. Correr las pruebas automáticas:

```
python -m pytest -q
```

7. (Opcional) Regenerar los casos propios y las tablas que usa el informe:

```
python generar_casos.py
python docs/generar_tablas.py
```

Para agregar casos nuevos basta dejar los archivos `.json` en `casos/` y volver a correr `python ejecutar_casos.py`: no hace falta cambiar el código.

El resultado esperado para el caso de la profesora es: aceleran las etapas E02, E03, E04, E06, E08 y E12; la duración pasa de 501 a 446 (ahorro 55, costo 72 de 72); y la cobertura de auditoría tiene 4 interacciones (E01, E16, E06, E03).

6 Limitaciones

- El DP de la Parte 1 es pseudopolinomial: su tiempo y su memoria crecen con el *valor* de B. Con presupuestos muy grandes (por ejemplo $n \cdot B$ de cientos de millones de celdas) puede tardar demasiado o quedarse sin memoria. Probado hasta $n = 500$ y $B = 5000$.
- Con flujos de rutas paralelas el método es exacto pero exponencial en el peor caso. Se probó con hasta 40 etapas (capas de ancho 8, cerca de un segundo). Con muchas más etapas puede alcanzar el tope de nodos; entonces devuelve la mejor solución hallada y lo avisa.
- El voraz de la Parte 2 no garantiza el óptimo; solo queda a un factor $H(d)$ de él.
- El verificador comprueba la optimalidad por enumeración solo en instancias pequeñas (cadenas con hasta unas 24 etapas candidatas, flujos generales con hasta 14, cobertura con hasta 22 etapas). En las grandes solo comprueba factibilidad y coherencia de las cifras.

- Esta entrega no incluye las variantes (auditoría ponderada, presupuesto conjunto, flujo con costos de auditoría) ni experimentos sistemáticos de escalamiento; quedan para la segunda entrega.
- Los nombres de clave alternativos (`estaciones`, `caminos`, `desde`, `hasta`) se aceptan por tolerancia, pero un formato con otras claves no se reconoce y produce un error de entrada explícito.